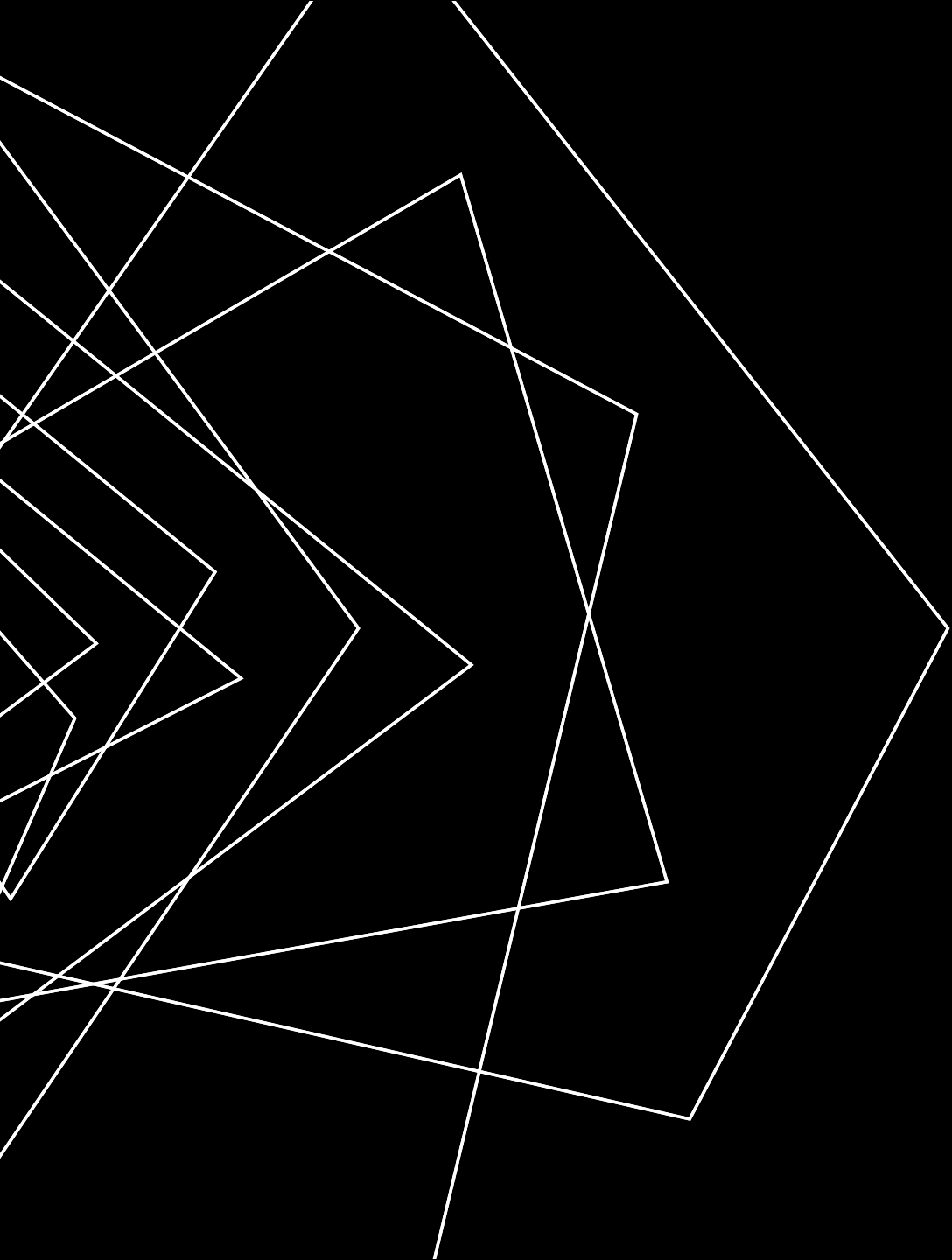




EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN



STATE & PROPS & HOOKS

Web Programming and Interface Design

LEARNING OBJECTIVES

- Learn component communication techniques
- Share state using lifting state up and Context API
- Understand project structure and best practices

REACTJS

- Component-Based Architecture: Reusable components for better maintainability.
- Declarative Syntax: Makes it easier to reason about your application.
- Virtual DOM: Efficient updates and rendering.
- Strong Community and Ecosystem: Rich set of tools and libraries.

KEY CONCEPTS

- Components: The building blocks of a React application.
- State: Manages dynamic data within a component.
- Props: Passes data from parent to child components.
- Lifecycle Methods: Hooks into different stages of a component's lifecycle.

REACT COMPONENT STRUCTURE

- Functional Components: Simple functions that return JSX.
- Class Components: ES6 classes with more features (less common in newer codebases).

JSX (JAVASCRIPT XML)

- **const element = <h1>Hello, world!</h1>;**

CREATING YOUR FIRST COMPONENT

SIMPLE COMPONENT

- Functional Component Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

- Usage:

```
<Welcome name="CSCI" />
```

PROPS AND STATE

- **Props:** Immutable data passed to components.
- **State:** Mutable data managed within the component.

PROPS AND STATE

- Example of State in a Functional Component using Hooks:

```
import React, { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>You clicked {count} times</p>

      <button onClick={() => setCount(count + 1)}>Click me</button>

    </div>

  );
}
```

LIFECYCLE METHODS

- Mounting: `componentDidMount()`
- Updating: `componentDidUpdate()`
- Unmounting: `componentWillUnmount()`

PROPS AND STATE

- Example of State in a Functional Component using Hooks:

```
import React, { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>You clicked {count} times</p>

      <button onClick={() => setCount(count + 1)}>Click me</button>

    </div>

  );
}
```

HANDLING EVENTS

```
function MyButton() {  
  function handleClick() {  
    alert('Button was clicked!');  
  }  
  
  return <button onClick={handleClick}>Click me</button>;  
}
```

LISTS AND KEYS

- Rendering Lists:

```
function NumberList(props) {  
  const numbers = props.numbers;  
  const listItems = numbers.map((number) =>  
    <li key={number.toString()}>{number}</li>  
  );  
  return (  
    <ul>{listItems}</ul>  
  );  
}
```

FORMS

- Controlled Components Example:

```
function NameForm() {  
  const [value, setValue] = useState("");  
  
  function handleChange(event) {  
    setValue(event.target.value);  
  }  
  
  function handleSubmit(event) {  
    alert('A name was submitted: ' + value);  
    event.preventDefault();  
  }  
}
```

FORMS

```
return (  
  <form onSubmit={handleSubmit}>  
    <label>  
      Name:  
      <input type="text" value={value} onChange={handleChange}  
/>  
    </label>  
    <button type="submit">Submit</button>  
  </form>  
);  
}
```

REACT ROUTER

- Purpose: Handling routing in single-page applications.

```
import { BrowserRouter as Router, Route, Routes } from 'react-router-dom';
```

```
function App() {  
  return (  
    <Router>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </Router>  
  );  
}
```

STATE MANAGEMENT LIBRARIES

- Overview: Tools for managing state across your application.
- Popular Libraries:
 - Redux
 - Context API
 - MobX

REACT HOOKS

- **What are Hooks?**

- Functions that let you use state and other React features without writing a class.

- **Common Hooks:**

- `useState()`
- `useEffect()`
- `useContext()`

COMPONENT COMMUNICATION

- Props: Parent to child data flow
- Lifting state up: Share state between components
- Context API: Avoid prop drilling, share global state

CONTEXT API BASICS

- Create Context: `const MyContext = React.createContext();`
- Provide Context with `<MyContext.Provider>`
- Consume Context with `useContext(MyContext)`

PROJECT STRUCTURE BEST PRACTICES

- Organize by features or components
- Separate UI and logic when possible
- Keep reusable components in /components folder
- Use clear naming conventions

DEPLOYMENT OPTIONS

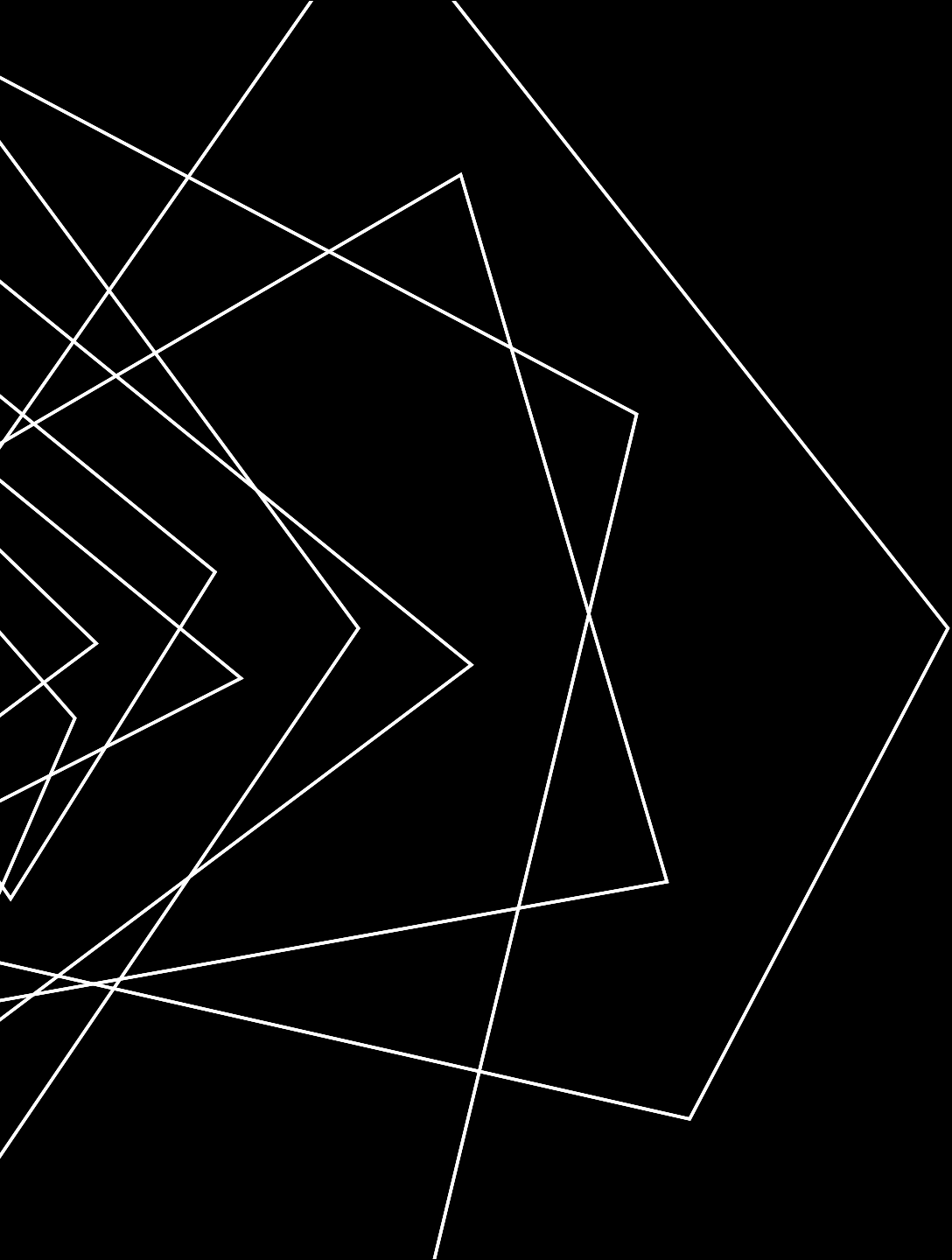
- Vercel: Easy integration with GitHub
- Netlify: Continuous deployment, free tier
- GitHub Pages: Static hosting option
- Build command: `npm run build`

ADVANCED TOPICS

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Code Splitting
- Error Boundaries

LAB EXERCISE

- Refactor previous apps into a single project
- Apply routing, state management, and styling



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335